# QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

## UMHackathon 2026

umhackathon@um.edu.my

---

© UMHackathon 2026

Email: umhackathon@um.edu.my

# Table of Content


**Document Control** **3**

**PRELIMINARY ROUND (Test Strategy & Planning)** **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7



*© UMHackathon 2026*



*Email: umhackathon@um.edu.my*

# Document Control

| Field | Detail |
|---|---|
| **System Under Test (SUT)** | *UMHackathon 2026 - Team **ms potato*** |
| **Team Repo URL** | *Jasminekhoo/ms-potato: UMHackathon 2026* |
| **Project Board URL** | - |
| **Live Deployment URL (optional for preliminary round)** | - |

## Objective:

*The primary objective of this system is to ensure that the AI Rental Advisor can reliably analyze property inputs, compute true rental costs, evaluate affordability and risk, and generate consistent AI-driven verdicts under concurrent usage, while maintaining acceptable latency, accuracy, and system stability.*

## PRELIMINARY ROUND (Test Strategy & Planning)

### 1. Scope & Requirements Traceability

*This section ensures all testing activities are aligned with system requirements through a **Requirement Traceability Matrix (RTM)**. It guarantees that each core feature is validated and reduces the risk of missing critical functionality or introducing unintended features.*

*Each feature is mapped to corresponding test cases to ensure full coverage and prevent untested system behaviour.*

#### 1.1 In-Scope Core Features

- **Property Analysis Input:** Users input property details (location/postcode, rent, income), with validation and backend processing.
- **AI Verdict Engine:** Generates classification (GREAT DEAL / ACCEPTABLE / AVOID) with natural language explanation using GLM.
- **True Cost Calculator:** Computes hidden costs (deposit, utilities, fees) and provides monthly breakdown.
- **Risk Radar:** Detects potential issues using keyword-based analysis and assigns a risk score with highlighted concern.
- **Property Comparison Mode:** Allows users to compare up to 3 properties side-by-side with

© UMHackathon 2026

Email: umhackathon@um.edu.my

structured outputs.

- **Market Benchmark Comparison:** Compares asking rent against postcode average using dataset and returns above/at/below market with percentage difference.

### 1.2 Out-of-Scope

- Advanced negotiation automation (auto messaging landlords)
- Interactive maps and geolocation features
- Long-term financial planning (e.g., home loan simulations)

## 2. Risk Assessment & Mitigation Strategy

*Quality assurance risks are identified based on system architecture, AI dependency, and integration complexity. Each risk is evaluated using the **5×5 Risk Matrix**.*

| Technical Risk | Likelihood (1–5) | Severity (1–5) | Risk Score (L×S) | Mitigation Strategy | Testing Approach |
|---|---|---|---|---|---|
| *AI API timeout or slow response (>8s)* | *4* | *4* | *16 (Critical)* | *Implement cached fallback responses and timeout handling* | *Simulate delayed API responses and verify fallback activation* |
| *Inconsistent AI verdicts for similar inputs* | *3* | *4* | *12 (High)* | *Combine deterministic rule-based scoring with controlled prompt design (low temperature) to ensure consistent outputs* | *Run repeated test cases with similar inputs and compare outputs* |
| *Incorrect cost calculation (hidden fees)* | *3* | *5* | *15 (High)* | *Validate formulas and use standardized assumptions* | *Unit testing for financial engine with edge cases* |
| *Invalid or unrealistic user input* | *4* | *3* | *12 (High)* | *Input validation (frontend + backend) and default constraints* | *Test boundary values (e.g., zero income, extreme rent)* |
| *Integration failure between modules* | *3* | *5* | *15 (High)* | *Use strict JSON schema and integration service* | *End-to-end testing across all modules* |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| Technical Risk | Likelihood (1–5) | Severity (1–5) | Risk Score (L×S) | Mitigation Strategy | Testing Approach |
|---|---|---|---|---|---|
| *Risk Radar false positives/negatives* | *3* | *3* | *9 (Medium)* | *Use weighted keyword scoring and clear disclaimers* | *Test with different keyword scenarios* |
| *Dataset inaccuracy (postcode pricing)* | *2* | *4* | *8 (Medium)* | *Label outputs as estimates and use % comparison* | *Compare dataset outputs with sample real listings* |
| *Frontend rendering failure due to incomplete API response* | *3* | *3* | *9 (Medium)* | *Implement fallback UI and default values* | *UI testing with incomplete/missing API responses* |
| *External API failure during demo scenario* | *4* | *5* | *20 (Critical)* | *Preload demo data and cached responses* | *Offline testing using cached responses only* |
| *AI hallucination (incorrect but convincing explanation)* | *3* | *5* | *15 (High)* | *Anchor AI outputs to computed backend data and restricts free-text generation* | *Compare AI explanations against actual calculated values* |

***Risk Assessment Scoring Criteria:***

| *Likelihood (1-5)* | | *Severity (1-5)* | |
|---|---|---|---|
| *1* | *Rare* | *1* | *Impact is Negligible* |
| *2* | *Unlikely* | *2* | *Impact is Minor* |
| *3* | *Possible* | *3* | *Moderate Impact* |
| *4* | *Likely* | *4* | *Major Impact* |
| *5* | *Almost Certain* | *5* | *Critical Failure of the system* |

***Risk Score = Likelihood × Severity***

***Risk Level Reference:***

| ***Risk Score*** | ***Risk Level*** | ***Recommended Action*** |
|---|---|---|
| *1 – 5* | *Low* | *Monitor only. Acceptable risks.* |
| *6 – 10* | *Medium* | *Mitigate + Testing* |
| *11 – 15* | *High* | *Must need mitigating and through testing is required* |

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

| | | |
|---|---|---|
| *16 – 25* | *Critical* | *Priority is Highest. Need extensive level of testing.* |

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

## 3. Test Environment & Execution Strategy

*This section describes the environment, tools, and strategies used to test the AI Rent Advisor system. The goal is to ensure that all components, including backend APIs, financial calculations, and AI integration, function correctly under different scenarios while maintaining system reliability and performance.*

- **Unit Test**
  - ***Scope:*** *Unit testing focuses on individual components such as:*
    - *Financial Risk Engine (cost calculation, affordability logic)*
    - *Market comparison logic (rent vs average)*
    - *Input validation (missing or invalid fields)*
  - ***Execution:*** *Tests are executed during development using manual testing and API testing tools such as Thunder Client or Postman. Each function is tested independently before integration.*
  - ***Isolation:*** *External dependencies such as the AI API (Z.ai GLM) are not required during unit testing. Mock data and predefined inputs are used to validate logic correctness.*
  - ***Pass Condition:***
    - *Functions return correct outputs for valid inputs*
    - *Edge cases (e.g., zero income, extremely high rent) are handled correctly*
    - *Output format is consistent and structured (valid JSON)*

- ***Integration Test***
  - ***Scope:*** *Integration testing ensures that all modules work together, including:*
    - *Backend API (`/api/analyse`)*
    - *Financial Risk Engine*
    - *AI module integration (GLM API calls)*
    - *Data processing and response formatting*
  - ***Execution:*** *Performed after unit testing is completed. Real API calls are made between:*
    - *Backend server and AI module*
    - *Backend and in-memory dataset*
  - ***Workflow:***
    1. *User submits property data*
    2. *Backend processes financial calculations*
    3. *AI API is called for verdict, cost, risk, and negotiation*
    4. *All results are merged into a single response*
  - ***Pass Condition:***
    - *All modules return valid data*
    - *API response contains all required fields (verdict, cost, risk, etc.)*
    - *No system crash during full flow execution*

- ***Test Environment (CI/CD Practice):***
  - ***Local Testing:*** *All testing is primarily conducted locally using:*
    - *Node.js backend server*
    - *Thunder Client / Postman for API testing*

© UMHackathon 2026

Email: umhackathon@um.edu.my

        - *Sample in-memory dataset*
    - ***Staging / CI:*** *Code is managed using GitHub. Team members push changes to branches and merge into the main branch after testing.*
    - ***CI/CD Automation:*** *Basic CI practices are followed:*
        - *Code is tested before merging*
        - *Integration testing is performed after combining modules*
        - *System stability is verified before demo deployment*

- ***Regression Testing & Pass/Fail Rules:***
    - ***Execution Phase:*** *Regression testing is performed whenever:*
        - *New features are added*
        - *Modules are integrated*
        - *Changes are merged into the main branch*
    - ***Pass/Fail Condition:***
        - *A test case is considered **Pass** when actual output matches expected output*
        - *A test case is considered **Fail** when output is incorrect, incomplete, or causes system errors*
    - ***Continuation Rule:***
        - *Critical features (e.g., `/api/analyse`) must pass all tests before proceeding*
        - *Failed cases must be fixed before further testing continues*

- ***Test Data Strategy:***
    - ***Manual Data:*** *The system uses a predefined in-memory dataset including:*
        - *Sample properties (location, rent, features)*
        - *User data (income, profiles)*
        - *Reviews and complaint records*
    - ***Automated Data:***
      *Test inputs are simulated using API requests with different scenarios:*
        - *Normal cases (valid property + income)*
        - *Edge cases (low income, high rent)*
        - *Invalid inputs (missing fields)*

- ***Passing Rate Threshold:***
    - *Minimum **85% overall test case pass rate. 100% pass rate required for critical features**, especially:`/api/analyse` endpoint, Financial calculations and API response structure. Failure to meet these thresholds requires bug fixing before further development or deployment.*

## 4. CI/CD Release Thresholds & Automation Gates

*This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.*

### *4.1 Integration Thresholds (Merging to Main)*

*The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.*

| ***Checks*** | ***Requirements*** | ***Project*** | ***Pass/Failed*** |
|---|---|---|---|
| ***Automatic Build*** | ***Zero Build Error*** | ***5 Errors*** | ***Passed*** |
| ***Unit Tests*** | ***≥ 95% passing rate*** | ***100%*** | ***Passed*** |
| ***Code Quality*** | ***≥ 98% clean (no critical issues)*** | ***97%*** | ***Failed*** |
| ***Test Coverage*** | ***Minimum 80%*** | ***79%*** | ***Passed*** |
| ***Integration Tests*** | ***All endpoints return valid JSON*** | ***100%*** | ***Passed*** |

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

***4.2 Deployment Thresholds (Pushing to Production)***

| ***Checks*** | ***Requirements*** | ***Project*** | ***Pass/Failed*** |
|---|---|---|---|
| ***Regression Test*** | ***≥ 90% pass rate*** | ***91%*** | ***Passed*** |
| ***AI Output Pass Rate*** | ***≥ 80% prompt consistency (validated outputs)*** | ***78%*** | ***Failed*** |
| ***Critical Bugs*** | ***Zero critical bugs*** | ***1*** | ***Failed*** |
| ***API Performance*** | ***Response time < 800ms (excluding AI latency)*** | ***520ms*** | ***Passed*** |
| ***GLM API Latency*** | ***< 8 seconds per request*** | ***6.5s avg*** | ***Passed*** |
| ***Fallback Reliability*** | ***Cache activates successfully when AI fails*** | ***100%*** | ***Passed*** |
| ***Security*** | ***No exposed API keys or secrets*** | ***Clean*** | ***Passed*** |

## 5. Test Case Specifications

| **Test Case ID** | **Test Type & Mapped Feature** | **Test Description** | **Test Steps** | **Expected Result** | **Actual Result** |
|---|---|---|---|---|---|
| *TC-01* | ***Happy Case (Entire Flow):*** *Comprehensive Analysis Result* | *Verify that the ResultScreen correctly renders all AI-generated components: Verdict, Cost Breakdown, Risk Radar, and Source* | *1. Submit property data via PropertyForm.*<br><br>*2. Observe* | *UI displays a clear "Verdict Card," a "Risk Radar," and a human-readable explanation from the GLM.* | *Successfully transitioned from skeleton to results. StaggeredReveal (index 0-4) animated* |

© UMHackathon 2026

| Test Case ID | Test Type & Mapped Feature | Test Description | Test Steps | Expected Result | Actual Result |
|---|---|---|---|---|---|
| | | *Transparency.* | *transition from LoadingSkeleton.*<br><br>*3. Verify all cards reveal via StaggeredReveal.*<br><br>*4. Check SourceTransparencyCard for data freshness.* | | *smoothly. All AI data points (Risk Score, Verdict, Logic) populated into the respective cards without UI overflow.*<br><br>*Status: Passed* |
| *TC-02* | ***Specific Case (Negative):*** *Incomplete AI Data Payload* | *Verify the UI doesn't crash if the **Z.ai GLM** returns a partial JSON.* | *1. Mock a response with result.negotiationTips as an empty list.*<br><br>*2. Navigate to /result.*<br><br>*3. Check the NegotiationCard.* | *The app handles the empty list gracefully. The card either shows "No tips available" or hides itself without throwing a null-check error.* | *NegotiationCard rendered an empty state message. The StaggeredReveal skipped the index smoothly. No app crash.*<br><br>*Status: Passed* |
| *TC-03* | ***NFR (Performance):*** *Perceived Latency (Skeleton UI)* | *Validate that the LoadingSkeleton provides enough visual feedback to prevent user abandonment during 8s+ AI processing.* | *1. Trigger analysis for a complex listing.*<br><br>*2. Start timer.*<br><br>*3. Observe UI* | *The LoadingSkeleton appears within 100ms. It must mimic the shape of the VerdictCard and RiskRadarCard to prevent layout shift.* | *Skeleton appeared instantly. Layout shift (CLS) was minimal when the real cards revealed. User perceived the wait as "AI Thinking."* |

© UMHackathon 2026

Email: umhackathon@um.edu.my

| Test Case ID | Test Type & Mapped Feature | Test Description | Test Steps | Expected Result | Actual Result |
|---|---|---|---|---|---|
| | | | *while isLoading == true.* | | *Status: Passed* |

## 6. AI Output & Boundary Testing (Drafts)

This section validates that the integration of **Z.ai GLM** produces acceptable outputs and handles abnormal inputs appropriately.

### 6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

| Test ID | Prompt Input | Expected Output (Acceptance Criteria) | Actual Output | Status |
|---|---|---|---|---|
| AI-01 | "Suggest a suitable rental based on budget RM800 in Kuala Lumpur." | Response includes relevant rental suggestions within budget, mentions location, no unrealistic or fabricated data | Provides general rental suggestions within budget range, mentions areas like Cheras/Subang | Pass |
| AI-02 | "Analyze this rental: RM1500, near LRT, 3 rooms. Is it worth it?" | Output gives a brief evaluation (2–4 sentences), considers price, location, and features, no hallucinated facts | Gives reasonable analysis based on input, highlights value and location advantage | Pass |

© UMHackathon 2026

Email: umhackathon@um.edu.my

**6.2. Oversized/Larger Input Test**

When the limit is exceeded, ILMU-GLM 5.1 prioritises the structured inputs (rent, location, income) over unstructured text (reviews, forum posts). The verdict and cost breakdown remain accurate; only supplementary review analysis may be incomplete.

| Fields | Details |
|---|---|
| Maximum Input Size | ~1500 tokens (approx. 800–1000 words) |
| Input used while testing | ~5000 words (property descriptions + user preferences + + forum reviews) |
| Expected Behavior | Input should be truncated or processed in chunks |
| Actual Behavior | System truncates input and still produces a partial but valid response |
| Status | Pass |

**6.3. Adversarial/Edge Prompt Test**

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

| Prompt Tested | "Just give me a place to rent, anywhere, cheap" |
|---|---|
| Issue | Ambiguous — no property name, location, or rent figure provided |
| GLM Response | Returns a prompt asking user to specify location and asking rent |
| System Handling | Input validation layer catches missing required fields before passing to GLM; user is shown a guided form to complete |
| Status | Pass |

**6.4. Hallucinating Handling**

*AI Rent Advisor mitigates hallucination through three layers:*

© UMHackathon 2026
Email: umhackathon@um.edu.my

- ***Grounded inputs*** *— GLM responses are anchored to structured data (postcode benchmarks, user-submitted rent figures) rather than generated from memory alone*
- ***Source-backed risk scores*** *— Risk Radar only flags issues when patterns appear across multiple sources; single complaints are filtered out to prevent false positives*
- ***Output validation*** *— Verdict responses are cross-checked against the benchmark dataset; if GLM suggests a price range that contradicts the dataset, the system defaults to the dataset value and flags the discrepancy*

© UMHackathon 2026

Email: umhackathon@um.edu.my